

Algoritmos - 2026-1

BCC2001

Diego Bertolini
diegobertolini@utfpr.edu.br

Aula 02

- **Aula Anterior:**

- **Sociedade e conceitos básicos de informática ;**

- **Aula de Hoje:**

- **Algoritmos: Lógica de programação e conceito de algoritmo.**
- **Tipos de representação de algoritmos.**
- **Método para a construção de algoritmos e resolução de problemas.**

Objetivo

O que vocês devem saber ao final da aula:

1) O que são Algoritmos.

1) Formas de representação.

2) Técnicas de criação e desenvolvimento de algoritmos.

Algoritmo

“Um conjunto finito de regras que provê uma sequência de operações para resolver um tipo de problema específico”

– [KNUTH]

(autor da coleção *“The art of computer programming”*)

Analogia com Receita

Um receita de um bolo é um algoritmo para resolver o problema:

“Como fazer um bolo”

Algoritmos devem ser precisos, entretanto receitas nem sempre são precisas, por exemplo, “colocar sal a gosto”.

E o resultado nem sempre é o mesmo, depende muito da pessoa que está cozinhando...

Origem

Segundo **Donald Knuth**, um papa na criação e estudo de algoritmos, a palavra tem origem no nome de um matemático persa, que viveu no século IX, cujo nome era:

Abu Ja'far Maomé ibn Mûsâ al-Khowârizm

ou

Pai de Já'far, Maomé, filho de Moisés da cidade de Khowârizm

www.lib.virginia.edu/science/parshall/khwariz.html

Características Importantes

1. **Finitude:** Um algoritmo deve sempre terminar após um número finito de passos.
2. **Definição:** Cada passo de um algoritmo deve ser precisamente definido. As ações devem ser definidas rigorosamente e sem ambiguidades.
3. **Entradas:** Um algoritmo deve ter zero ou mais entradas, fornecidas antes do algoritmo iniciar.
4. **Saídas:** Um algoritmo deve ter uma ou mais saídas.
5. **Efetividade:** Todas as operações devem ser simples de modo que possam ser executadas em um tempo limitado.

Dificuldades iniciais

- Difícil para iniciantes saber o que o computador pode ou não fazer.
- Criação de algoritmos não é um processo automático, cada pessoa tem uma forma de pensar, de organizar o raciocínio.
- Pode haver mais de uma solução para um problema.

Algoritmo

O algoritmo deve ensinar a resolver o problema, e não trazer a solução.

Exemplo: receita de bolo.

- Ação é um acontecimento que, a partir de um estado inicial, após um período de tempo finito, produz um resultado final previsível e bem definido.-
- Objetivo: **solução do problema.**

Algoritmo

- **Aspecto lógico: ordenar o pensamento.**

- Organização sequencial das ações pertinentes à solução de um problema;

- **Exemplo:** o bolo só pode ser levado ao forno depois que a massa estiver pronta.

- Lógica no dia-a-dia:

- Todo aluno estuda bastante.

- Vocês são alunos.

- Portanto, vocês estudam bastante.

Metodologia

- Passo 1: Análise do Problema
- Passo 2: Projeto do Programa
- Passo 3: Implementação
- Passo 4: Verificação

Passo 1: Análise do Problema

- Compreensão correta do problema
- Entendimento das entradas e saídas

Passo 2: Projeto do Programa*

- Desenvolver um ALGORITMO
- Definir uma estrutura de dados
- Otimizar o binômio tempo – espaço

Passo 3: Implementação

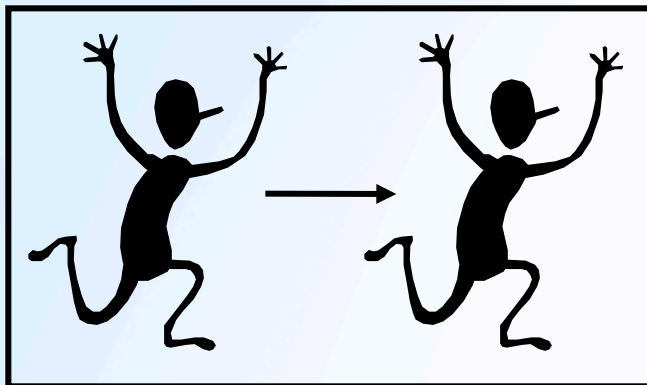
- Codificação do ALGORITMO em uma linguagem de programação

Passo 4: Verificação

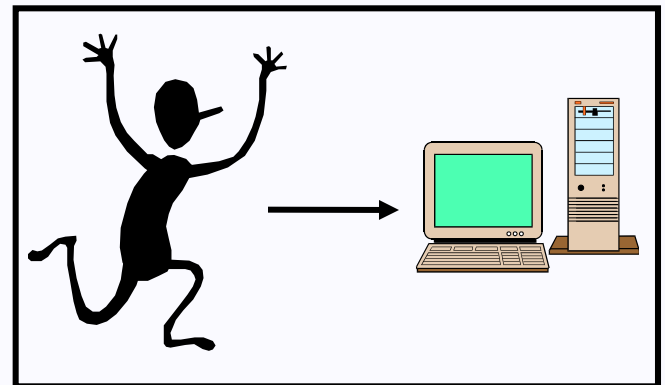
- Demonstrar que o algoritmo realmente resolve o problema proposto, qualquer que seja sua instância

Algoritmos

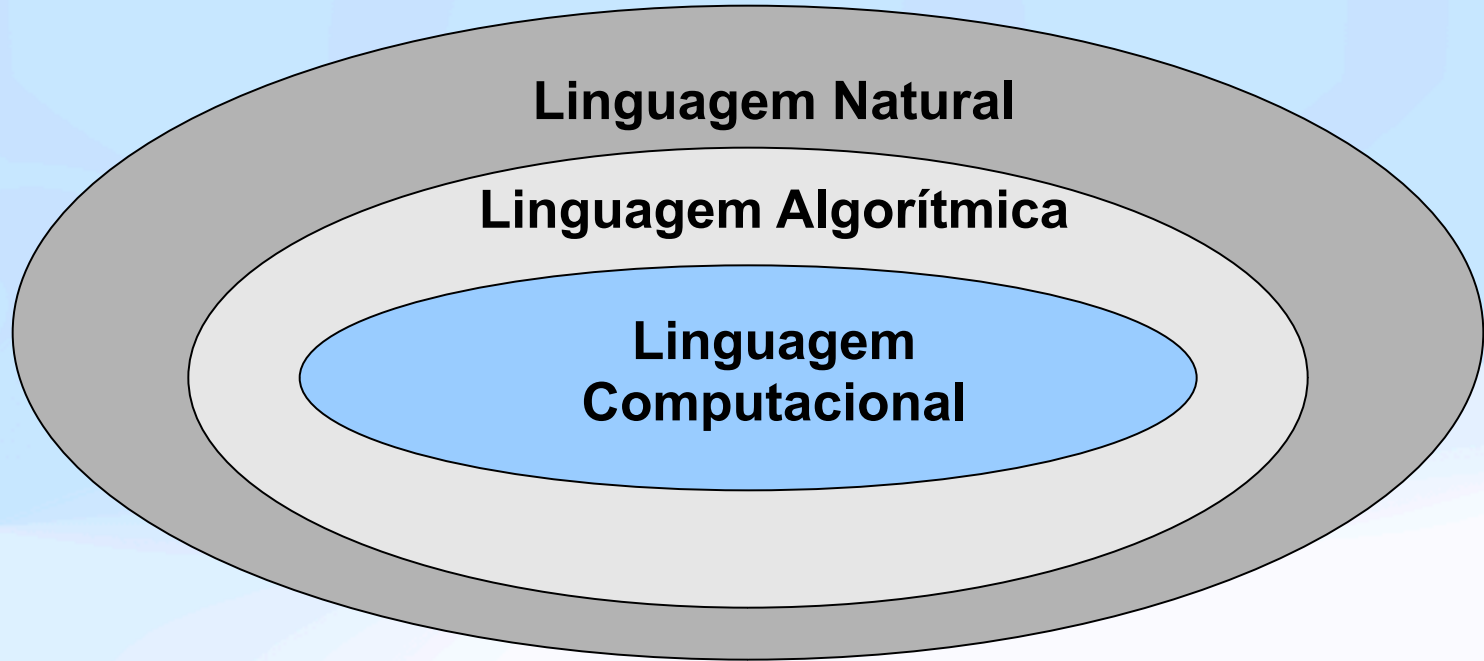
- **Ambiguidade:** relacionada à sintaxe e semântica de linguagens naturais.
- **“O artista produz formas perfeitas.”**
 - Linguagens naturais são muito extensas, possuindo ambiguidade;
 - Linguagem Natural X Linguagem Computacional



X



Algoritmos



Nos algoritmos, devemos utilizar construções que executarão uma ação específica bem definida;

- Uma linguagem algorítmica deve ter sintaxe rígida e semântica bem definida;
- Toda linguagem possui sintaxe e semântica.

Algoritmos

- **Sintaxe:** Dita as regras de como as sentenças e cada um de seus elementos devem ser construídos corretamente;
 - Regras Sintáticas, forma de escrever o código, como escrever comandos, onde usar parênteses, ponto e vírgula, como declarar variáveis.
- **Semântica:** se preocupa com o significado de uma sentença construída.
 - O que o código realmente faz, significado, comportamento do código.

Algoritmos

- **A sintaxe por si só não define completamente uma linguagem.**
 - **Exemplo: “Descoloridas ideias verdes sonham furiosamente”**
(Chomsky)
 - **Por outro lado: “Nóis foi, mais já vortemo”**

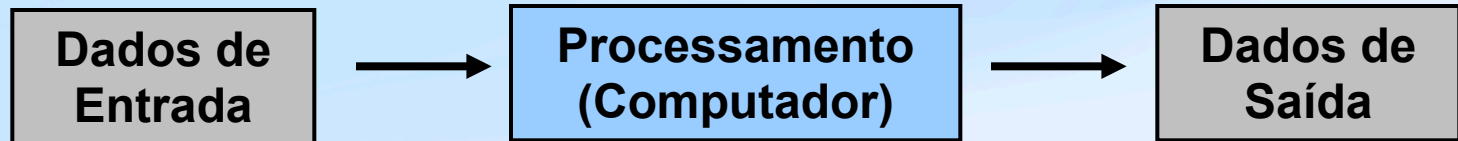
Outro exemplo:

O cachorro comeu o osso. O osso comeu o cachorro.

Int idadePessoa = -200 ;

Algoritmos

- Algoritmos geralmente descrevem algum processamento de dados:



- Estes dados precisam ser representados (descritos) nos algoritmos;

Algoritmos

- Um algoritmo para fins computacionais é a descrição de ações que manipulam objetos em função do tempo.
 - Objetos descrevem o estado computacional;
 - Objetos têm função específica;
 - Ações são direcionadas aos objetos;
 - Objetos são definidos como parte do programa;
 - Estados diferentes requerem objetos diferentes.
- * Objetos frequentemente serão chamados de variáveis.

Algoritmos

- A cada objeto é associado um nome que identifica este objeto ao longo da programação.
- Este nome é chamado ***identificador***.
- De forma geral, podemos dizer que um ***identificador*** está ***associado à uma célula de memória***.
- O termo ***variável*** é frequentemente utilizado como ***sinônimo de identificador***.

Algoritmos

- **Em geral, as linguagens de programação exigem nomes de identificadores com as seguintes características:**

- Use somente letras e números;
- O primeiro caracter do nome deve ser uma letra;
- Não é permitido o uso de caracteres especiais.

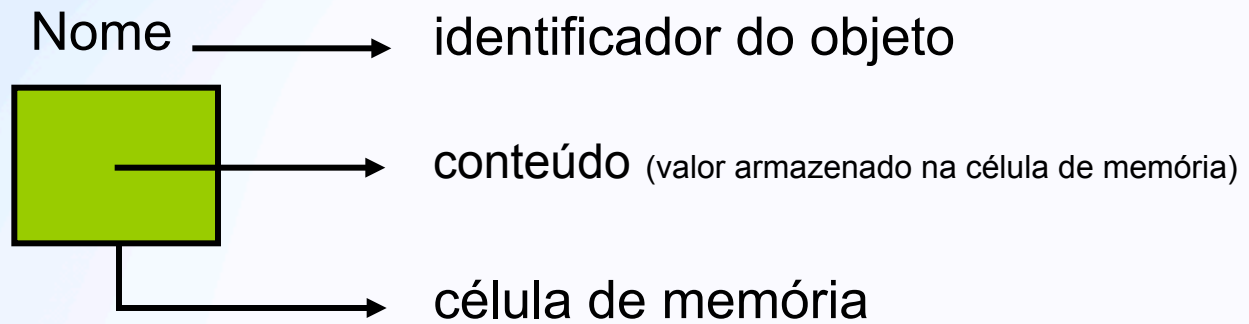
- **Exemplos válidos:** ImpostoDeRenda, F1, soma, produto, desconto, SalarioHora.

- **Exemplos inválidos:** 1f, Imposto-renda, salario hora.

Algoritmos

•Atributos (características) de um objeto:

- Nome;
- Célula de memória;
- Valor associado;
- Tipo: define a categoria dos dados que podem ser armazenados no objeto.



Algoritmos

- **O *tipo*:**

- especifica uma classe de valores que podem ser armazenados na célula de memória associada ao identificador;
- define as operações válidas sobre o identificador;

Algoritmos

- **Tipo de Dado**

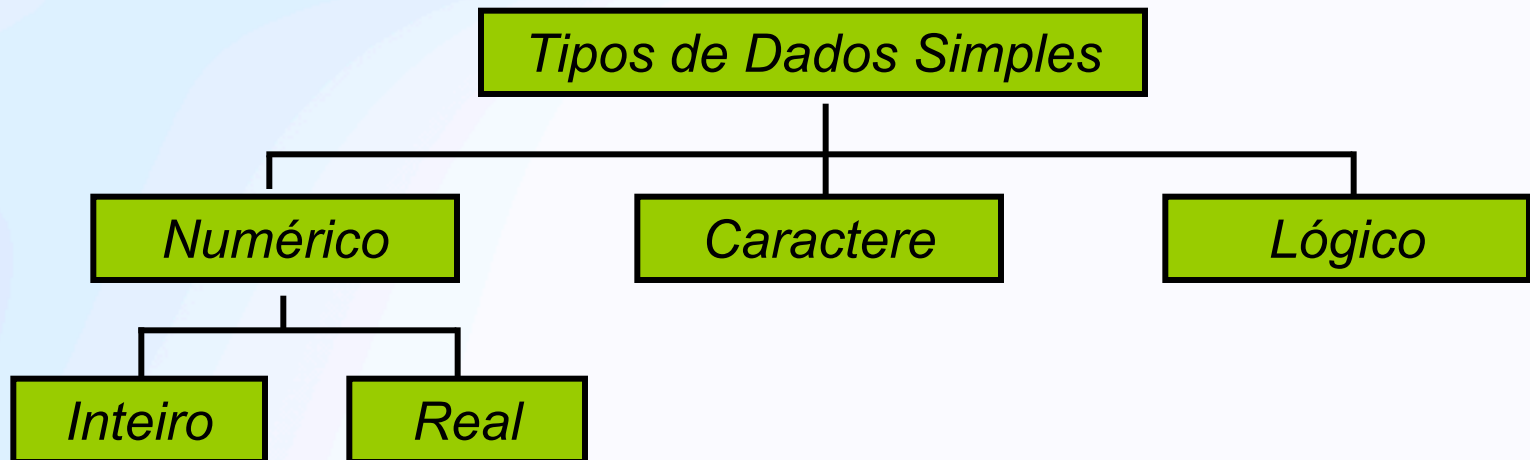
- Um identificador armazena um único dado num instante, mas este dado não pode ser qualquer um;
- Os objetos serão classificados segundo um tipo;
- Os dados podem ter estruturas físicas diferentes;
- As células de memória do computador precisam ser compatíveis com o tipo de dado que vão armazenar;

Algoritmos

- Devemos informar ao computador qual o tipo de dado que vamos armazenar;
- Esta informação determina quantas células de memória serão necessárias para armazenar o dado;
- Esta informação define quais serão as operações possíveis de se aplicar sobre os dados;
 - Exemplo: em um dado numérico pode-se aplicar uma operação aritmética.

Algoritmos

- Os tipos de dados podem variar de uma linguagem para outra.
- Existem dois tipos básicos:
 - Tipos simples;
 - Tipos estruturados;



Algoritmos

- Os Tipos de Dados Numéricos são divididos basicamente em dois grandes conjuntos:

- Inteiros;

- Reais;

- Os inteiros podem ser positivos, negativos ou nulos, mas não possuem um componente decimal;

Algoritmos

- Os inteiros são compatíveis com os reais, mas os reais não são compatíveis com os inteiros;
- Assim, um objeto real pode receber um valor inteiro, mas um objeto inteiro não pode receber um valor real;

Algoritmos

- Os reais podem ser positivos, negativos ou nulos, e possuem sempre um componente decimal;
- **Exemplos:** 2,34; 0,0; -214,123;
- Computacionalmente, existem muitas diferenças entre o armazenamento de números inteiros e números reais;

Algoritmos

- **Os tipos de dados não numéricos:**

- **Caractere:** formado pelas letras (de *A* a *Z*, e de *a* a *z*), dígitos (de 0 a 9) e caracteres especiais ($\sim$, $\cdot$, $?$, $>$, $<$, ...);

- A diferença entre caracteres e dados numéricos estão na forma de armazenamento, e nas operações legais permitidas sobre os mesmos;

- Exemplo: operações aritméticas sobre os dados numéricos.

Algoritmos

- Objetos declarados como do tipo caractere poderão armazenar um único caractere;
- Para armazenar uma sequência de caracteres será utilizada a estrutura cadeia de caracteres, que consiste num tipo de dado estruturado;

–Exemplo:

- ‘Z’ é um caractere;
- ‘ANA’ é uma cadeia de caracteres;

Algoritmos

- **Tipo Lógico:** os valores lógicos podem assumir uma entre duas possibilidades (verdadeiro ou falso);
 - Só poderá armazenar um destes dois valores;
- **Muito utilizados em controle do fluxo lógico do algoritmo;**

Algoritmos

- **Escolha do tipo de um identificador: deve-se conhecer bem a solução do problema.**

- **Exemplos de inteiros:**

- número de pessoas;
- um número par ou ímpar;
- idade de uma pessoa;
- quantidade de objetos;

- **Exemplos de reais:**

- o peso de uma pessoa;
- um percentual;
- nota de uma avaliação;
- média;
- preço de um produto;

Algoritmos

- **Situações de dúvida**

- **Exemplos:**

- raiz quadrada de um número;
 - divisão entre dois números inteiros;

- **Quando houver alguma possibilidade de um identificador numérico receber um valor não inteiro, deve ser declarado como real.**



Algoritmos

- **Argumentos de Entrada e Saída**

- **Entrada**: objetos cujos valores são conhecidos em tempo de execução;

- **Saída**: objetos que armazenam a solução objetivo de um problema;

Formas de Representação

- Descrição em Linguagem Natural
- Fluxograma
- Diagrama de Nassi-Schneiderman ou de Chapin
- Linguagem Algorítmica ou Pseudocódigo ou Portugol
- Linguagem de Programação

Linguagem de Programação

- A própria linguagem de programação é uma forma de representação de algoritmos.
- Neste caso os algoritmos estão codificados na linguagem de programação específica.

Exemplo

```
media.c ✕
1 #include <stdio.h>
2
3 int main(){
4     float n1, n2, m;
5
6     printf("Digite as duas notas:\n");
7     scanf("%f", &n1);
8     scanf("%f", &n2);
9
10    m = (n1 + n2) / 2;
11
12    printf("Média: %.2f\n", m);
13    if(m >= 7.0){
14        printf("Aprovado\n");
15    }
16    else{
17        printf("Reprovado\n");
18    }
19 }
```

Técnicas de Construção

- **Divisão para a conquista:** Técnica de divisão do problema em partes menores.
- As soluções das partes menores combinam-se para a composição da solução global.
- **Exemplo:** receita de um bolo (modo de preparo do bolo, preparo do recheio, preparo da cobertura)
- Permite que seja preparado por mais de uma pessoa e simplifica a execução
- **Tarefas independentes → paralelismo.**

Técnicas de Construção

- **Reaproveitamento:** Reutilização de soluções como parte de uma nova solução.
- **Exemplo:** Algoritmo de validação de CPF. Utilizado em todo e qualquer sistema que precise desse tipo de validação.
- Em um livro de receitas, os pratos que necessitassem o preparo de um molho branco, poderiam referenciar a página da receita do molho branco.
“Para o preparo do molho leia a página 25”
- **O livro fica menor, alterações em apenas um lugar.**

Dúvidas?